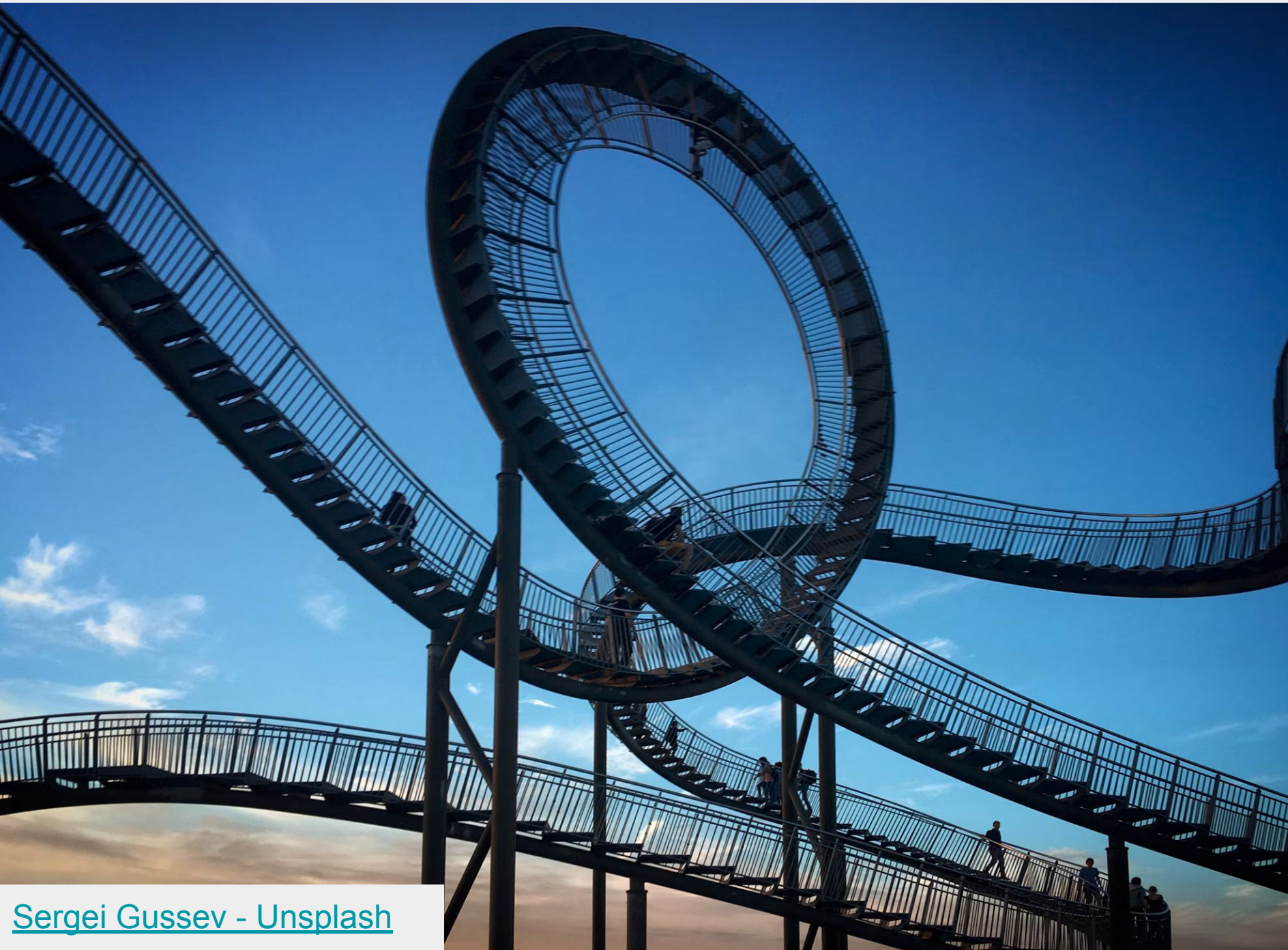
bash scripting – loops

Adapted from materials by Dr. Carrier



Sergei Gussev - Unsplash

for loops

Do we expect C-style or Python-style for loops?

for loops

Do we expect C-style or Python-style for loops?

```
for(int i = 0; i < 10; i++){ ... }


vs


for i in range(10):
```

for loops

More like Python!

for loops

More like Python!

```
for var in list
do
   ...
done
```

for loops

More like Python!

```
for var in list
do
   ...
done
```

```
for var in 1 2 3 4 5
for var in “alice” “bob” “carol”
for idx in {0..10..2}
for var in $@
```

A note on syntax

Recall: `$(command)` will run the command inside

A note on syntax

Recall: `$(command)` will run the command inside

```
var=$(ls)


for idx in $(seq 0 10)
do
   …
done
```

A note on syntax

Recall: `$(command)` will run the command inside

```
var=$(ls)


for idx in $(seq 0 10)
do
   …
done
```



while loops

```
while condition
do
   …
done
```

while loops

```
while condition
do
   …
done
```

Can use either syntax:

```
while [ ! -e output.txt ]
```

```
while (( x < 10 ))
```

What are our conditions? Same as `if`

Don’t worry about memorizing these:

https://ryanstutorials.net/bash-scripting-tutorial/bash-if-statements.php

Operator	Description
! EXPRESSION	The EXPRESSION is false.
-n STRING	The length of STRING is greater than zero.
-z STRING	The lengh of STRING is zero (ie it is empty).
STRING1 = STRING2	STRING1 is equal to STRING2
STRING1 != STRING2	STRING1 is not equal to STRING2
INTEGER1 -eq INTEGER2	INTEGER1 is numerically equal to INTEGER2
INTEGER1 -gt INTEGER2	INTEGER1 is numerically greater than INTEGER2
INTEGER1 -lt INTEGER2	INTEGER1 is numerically less than INTEGER2
-d FILE	FILE exists and is a directory.
-e FILE	FILE exists.
-r FILE	FILE exists and the read permission is granted.
-s FILE	FILE exists and it's size is greater than zero (ie. it is not empty).
-w FILE	FILE exists and the write permission is granted.
-x FILE	FILE exists and the execute permission is granted.

`break` and `continue`

They work exactly as expected.

- `break` will end the loop
- `continue` skips this iteration